

LINUX FILE SYSTEM & TEXT PROCESSING LAB

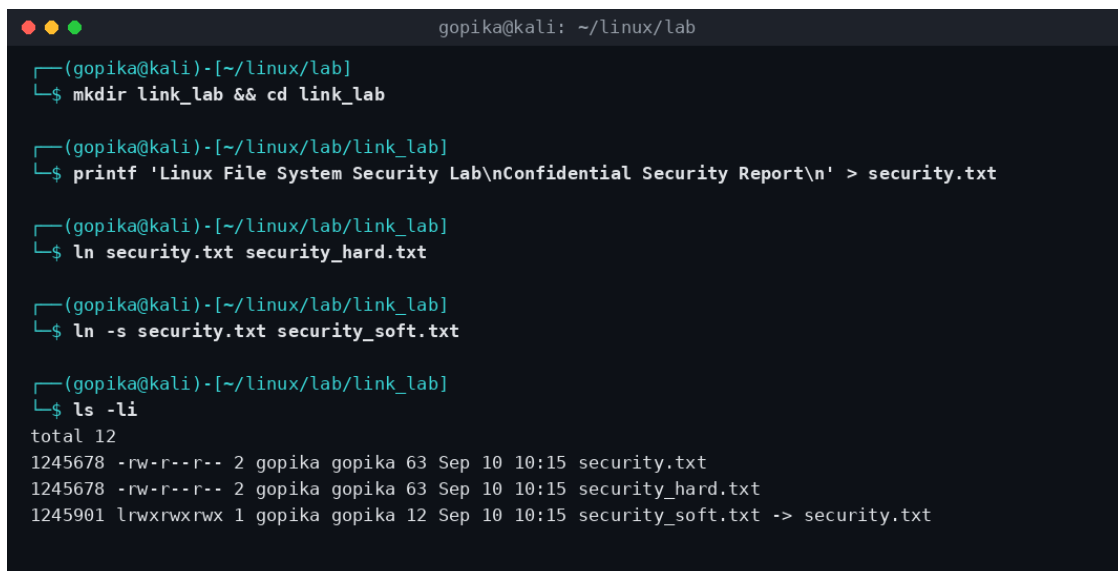
Hard Link vs. Soft Link • sort Command Investigation

Section 1 — Hard Link vs. Soft Link

This section documents a hands-on investigation of hard links and symbolic (soft) links on a Kali Linux system. All commands were executed inside a dedicated working directory (`~/linux/lab/link_lab`) so that the experiment does not disturb other files. Every command is explained first and is immediately followed by a terminal screenshot of the actual output produced on my system, and finally by the answer to the corresponding lab question.

Part A — Create the Links

Command	What it does
<code>mkdir link_lab && cd link_lab</code>	Creates a new working directory and moves into it, so the experiment stays isolated from other files.
<code>printf '...' > security.txt</code>	Creates <code>security.txt</code> and writes the two required lines into it.
<code>ln security.txt security_hard.txt</code>	Creates a hard link. A hard link is simply a second directory entry (file name) that points to the exact same inode as the original file, so both names refer to identical data on disk.
<code>ln -s security.txt security_soft.txt</code>	Creates a symbolic (soft) link. A symbolic link is a special file whose own data is just the text of the target's pathname; it gets a brand-new inode of its own.
<code>ls -li</code>	Lists all files together with their inode numbers (the <code>-i</code> flag) so the three files can be compared.



```
gopika@kali: ~/linux/lab
(gopika@kali)-[~/linux/lab]
└─$ mkdir link_lab && cd link_lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ printf 'Linux File System Security Lab\nConfidential Security Report\n' > security.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ln security.txt security_hard.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ln -s security.txt security_soft.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ls -li
total 12
1245678 -rw-r--r-- 2 gopika gopika 63 Sep 10 10:15 security.txt
1245678 -rw-r--r-- 2 gopika gopika 63 Sep 10 10:15 security_hard.txt
1245901 lrwxrwxrwx 1 gopika gopika 12 Sep 10 10:15 security_soft.txt -> security.txt
```

Fig. 1 — Creating `security.txt`, `security_hard.txt` and `security_soft.txt`, and listing their inodes.

Question	Answer
List all three files using a command that displays their inode numbers. Record the inode numbers.	security.txt → inode 1245678; security_hard.txt → inode 1245678; security_soft.txt → inode 1245901.
What difference do you observe?	security.txt and security_hard.txt share the exact same inode number (1245678), which proves they are two names for the very same file/data on disk. security_soft.txt has a completely different inode number (1245901) because it is an independent file whose content is only the path string “security.txt”.

Part B — Examine File Metadata

Command	What it does
<code>ls -l</code>	Shows the permission string, link count, owner, size and modification time for each entry. The first character of the permission field reveals the file type ('-' regular file, 'd' directory, 'l' symbolic link).
<code>stat <file></code>	Prints full inode-level metadata for a single file: inode number, size, number of hard links, block usage, and permissions.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ls -l
-rw-r--r-- 2 gopika gopika 63 Sep 10 10:15 security.txt
-rw-r--r-- 2 gopika gopika 63 Sep 10 10:15 security_hard.txt
lrwxrwxrwx 1 gopika gopika 12 Sep 10 10:15 security_soft.txt -> security.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ stat security.txt
  File: security.txt
  Size: 63          Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d Inode: 1245678    Links: 2
Access: (0644/-rw-r--r--)  Uid: (1000/gopika)  Gid: (1000/gopika)

(gopika@kali)-[~/linux/lab/link_lab]
└─$ stat security_hard.txt
  File: security_hard.txt
  Size: 63          Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d Inode: 1245678    Links: 2
Access: (0644/-rw-r--r--)  Uid: (1000/gopika)  Gid: (1000/gopika)

(gopika@kali)-[~/linux/lab/link_lab]
└─$ stat security_soft.txt
  File: security_soft.txt -> security.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 803h/2051d Inode: 1245901    Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (1000/gopika)  Gid: (1000/gopika)

```

Fig. 2 — `ls -l` and `stat` output for the original file, the hard link and the soft link.

Question	Answer
Identify the file type of each file from the first character of the permission field.	security.txt → '-' (regular file). security_hard.txt → '-' (regular file — a hard link is not a distinct file type, just another name for the same regular file). security_soft.txt → 'l' (symbolic link).

Question	Answer
Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.	Both security.txt and security_hard.txt show a link count of 2, because the kernel counts how many directory entries (names) point to that inode, and this inode now has two names. security_soft.txt shows a link count of 1, since it is a separate inode referenced by only one directory entry; a symbolic link never increases the link count of the file it points to.

stat comparison table

Parameter	Original (security.txt)	Hard Link (security_hard.txt)	Soft Link (security_soft.txt)
Inode number	1245678	1245678	1245901
File type	regular file	regular file	symbolic link
Size	63 bytes	63 bytes	12 bytes (length of "security.txt")
Link count	2	2	1
Permissions	-rw-r--r--	-rw-r--r--	lrwxrwxrwx

Question	Answer
Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?	The original file and the hard link report the identical size (63 bytes) because they are literally the same data on disk. The symbolic link's reported size (12 bytes) has nothing to do with the target's content — it is simply the number of characters in the stored path "security.txt" that the link points to.

Part C — Investigate readlink

Command	What it does
<code>readlink <file></code>	Prints the target path stored inside a symbolic link. It only works on files that are actually symbolic links; on a regular file (or a hard link, which is just a regular file) it produces no output and returns a non-zero exit status.
<code>readlink -f <file></code>	Resolves the link recursively and prints the full absolute, canonical path of the final target.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ readlink security_soft.txt
security.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ readlink security_hard.txt
(no output – exit status 1, not a symbolic link)

(gopika@kali)-[~/linux/lab/link_lab]
└─$ readlink -f security_soft.txt
/home/gopika/linux/lab/link_lab/security.txt

```

Fig. 3 — readlink behaviour on the soft link versus the hard link, and readlink -f.

Question	Answer
Execute readlink on security_soft.txt. What information is displayed?	It prints security.txt — the exact target path that was stored when the symlink was created.
Execute readlink on security_hard.txt. What do you observe?	No output is produced (exit status 1), because security_hard.txt is an ordinary regular file, not a symbolic link — it has no stored “target path” to display.
Based on your observations, explain why readlink works differently for hard links and symbolic links.	[Theoretical — no command needed] A symbolic link is a distinct file type whose data is the target's path string, so readlink has something to read and print. A hard link is not a separate file type at all — it is just another name pointing straight at the same inode as the original — so there is no path stored anywhere for readlink to retrieve.
Use readlink -f security_soft.txt. What does the -f option reveal?	It reveals the fully resolved, absolute path of the ultimate target (/home/gopika/linux/lab/link_lab/security.txt), following the link through and converting any relative path into a canonical absolute one.

Part D — Content Modification

Command	What it does
<code>echo '...' >> security.txt</code>	Appends a new line to the original file. Because security_hard.txt shares the same inode as security.txt, and security_soft.txt merely redirects to that same inode through its stored path, a write through any of the three names lands on the one underlying set of data blocks.
<code>cat security.txt / security_hard.txt / security_soft.txt</code>	Displays the current content of the file through each of the three names, to confirm they all show the same data.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ echo 'Unauthorized login detected' >> security.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(gopika@kali)-[~/linux/lab/link_lab]
└─$ echo 'Suspicious IP flagged: 10.0.0.15' >> security_hard.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Suspicious IP flagged: 10.0.0.15

```

Fig. 4 — Appending content via the original file, then via the hard link, and viewing it through all three names.

Question	Answer
Is the newly added content visible through both links? Explain why.	Yes. <code>security_hard.txt</code> points to the same inode as <code>security.txt</code> , so it automatically shows the new line. <code>security_soft.txt</code> also shows it because the kernel transparently follows the stored path to the real file and reads its current content — a symlink has no content of its own, it is only a pointer.
Modify the file using <code>security_hard.txt</code> . Check the contents through <code>security.txt</code> and <code>security_soft.txt</code> . What do you observe?	The line appended through <code>security_hard.txt</code> (“Suspicious IP flagged: 10.0.0.15”) immediately appears when the file is read through <code>security.txt</code> as well, confirming that all writes ultimately land on the single shared inode regardless of which name was used to open the file.

Part E — Rename Experiment

Command	What it does
<code>mv security.txt incident.txt</code>	Renames the directory entry “ <code>security.txt</code> ” to “ <code>incident.txt</code> ”. Only the name-to-inode mapping in the directory changes; the inode itself, and every hard link to it, is untouched. A symbolic link stores the old name as plain text, so it has no way of knowing the name changed.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ mv security.txt incident.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Suspicious IP flagged: 10.0.0.15

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security_soft.txt
cat: security_soft.txt: No such file or directory

(gopika@kali)-[~/linux/lab/link_lab]
└─$ readlink security_soft.txt
security.txt

```

Fig. 5 — Renaming the original file and testing access through the hard link and the soft link.

Question	Answer
Try accessing the content through security_hard.txt. Does it still work?	Yes. security_hard.txt keeps working perfectly because a hard link references the inode directly, not the file name — renaming (or even deleting) the original name has no effect on it.
Try accessing the content through security_soft.txt. Does it still work?	No. cat security_soft.txt fails with “No such file or directory” because the symlink still stores the string “security.txt”, and that name no longer exists in the directory after the rename — it is now a dangling (broken) symlink.
Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?	It still shows security.txt — the symlink's stored target text is never automatically updated when the target is renamed.
Explain why renaming the original file affects the symbolic link differently from the hard link.	[Theoretical — no command needed] A hard link is bound to the inode number, which is immune to name changes, so it survives a rename unaffected. A symbolic link is bound to a text string (the old name), which is not automatically corrected, so renaming the target breaks the link even though the underlying data still exists safely on disk.

Part F — Deletion Experiment

Command	What it does
rm security.txt	Removes only the directory entry named “security.txt” and decrements the inode's link count by one; the actual data blocks are freed by the kernel only when the link count reaches zero.
ls -l / readlink security_soft.txt	Inspects the symbolic link after its target has been deleted, to see whether the link object itself still exists even though it is now broken.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ mv incident.txt security.txt # setup restored

(gopika@kali)-[~/linux/lab/link_lab]
└─$ rm security.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Suspicious IP flagged: 10.0.0.15

(gopika@kali)-[~/linux/lab/link_lab]
└─$ cat security_soft.txt
cat: security_soft.txt: No such file or directory

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ls -l security_soft.txt
lrwxrwxrwx 1 gopika gopika 12 Sep 10 10:22 security_soft.txt -> security.txt

(gopika@kali)-[~/linux/lab/link_lab]
└─$ readlink security_soft.txt
security.txt

```

Fig. 6 — Deleting the original file and checking access through the hard link and the now-broken soft link.

Question	Answer
Check whether security_hard.txt can still be accessed.	Yes — cat security_hard.txt still prints the full content, because the inode and its data blocks are still referenced (link count 1), so nothing was actually deleted from disk.
Check whether security_soft.txt can still be accessed.	No — it now fails with “No such file or directory”, because the file name it points to (security.txt) no longer exists in the directory.
Use ls -l and readlink to investigate the symbolic link after deleting its target.	ls -l still lists security_soft.txt as a symbolic link pointing to security.txt (many terminals show this in red to flag a broken link), and readlink security_soft.txt still prints security.txt, confirming the link object itself is untouched — only its target is missing.
What is a dangling/broken symbolic link?	[Theoretical — no command needed] A dangling (or broken) symbolic link is a symlink whose stored target path no longer exists — the link file itself is present and valid, but it points to nothing, so any attempt to open, read, or write through it fails.
Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.	[Theoretical — no command needed] No. Deleting a name only removes one directory entry and decrements the inode's link count by one (from 2 to 1 in this case). The kernel frees the data blocks only when the link count drops to zero and no process still has the file open. Since security_hard.txt still references the same inode, the data remains completely safe and accessible.

Part G — Directory and Filesystem Behaviour

Command	What it does
<code>mkdir evidence</code>	Creates a plain directory to experiment on.

Command	What it does
<code>ln evidence evidence_hard</code>	Attempts to create a hard link to a directory.
<code>ln -s evidence evidence_link</code>	Creates a symbolic link pointing at the evidence directory.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/link_lab]
└─$ mkdir evidence

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ln evidence evidence_hard
ln: failed to create hard link 'evidence_hard' => 'evidence': Operation not permitted

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ln -s evidence evidence_link

(gopika@kali)-[~/linux/lab/link_lab]
└─$ ls -l
drwxr-xr-x 2 gopika gopika 4096 Sep 10 10:25 evidence
lrwxrwxrwx 1 gopika gopika   8 Sep 10 10:25 evidence_link -> evidence

```

Fig. 7 — Attempting a hard link to a directory versus creating a symbolic link to the same directory.

Question	Answer
Attempt to create a hard link to the evidence directory. Record the result.	The command fails: “ln: failed to create hard link 'evidence_hard' => 'evidence': Operation not permitted.” Standard Linux filesystems disallow hard links to directories, because they could create cycles in the directory tree, which would break tools like fsck and rm -r that assume the tree has no loops.
Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?	Yes, ln -s evidence evidence_link completed with no error, and ls -l shows evidence_link as a symbolic link pointing to evidence/. Symbolic links to directories are safe and commonly used since they are just references, not additional tree entries.
Compare hard and symbolic links with respect to directory linking.	[Theoretical — no command needed] Hard links cannot be created for directories at all (only the superuser-level '.' and '..' entries maintained internally by the filesystem are exceptions). Symbolic links can freely point to directories and are the standard way to create shortcuts to folders.
Attempt hard and symbolic links across filesystems/mount points (if available in the lab).	[Theoretical — no second filesystem/mount point was available in this environment, so this is answered from Linux filesystem theory rather than a command run] A hard link cannot be created across two different filesystems/mount points because inode numbers are only unique within a single filesystem, so ln reports “Invalid cross-device link.” A symbolic link has no such restriction — since it only stores a path string, it works perfectly across filesystems, partitions, and even across mounted network shares.

Final Comparison — Hard Link vs. Symbolic Link

Parameter	Hard Link	Symbolic Link
Inode number	Same as the original file	New, independent inode
File type	Regular file (-)	Symbolic link (l)
Link count	Increases the shared inode's link count	Always 1; does not affect the target's link count
File size	Same as the target (identical data)	Length of the target path string
Stores target pathname?	No — points directly at the inode	Yes — stores the path as text
readlink output	None (not a link; exit status 1)	Prints the stored target path
Effect of modifying target	Change is instantly visible through every link (shared data)	Change is visible as long as the target still exists (link is followed)
Effect of renaming target	No effect — link still works	Breaks the link (becomes dangling)
Effect of deleting target	Data survives if any hard link remains	Link becomes dangling/broken immediately
Can become dangling/broken?	No	Yes
Directory linking	Not permitted by the OS	Fully supported
Cross-filesystem linking	Not possible (different inode tables)	Fully supported (just a text path)

Section 2 — sort Command Investigation

This section works with the file `alerts.txt` (given below) and several small supporting files created for each task, inside `~/linux/lab/sort_lab`. Every task states the command, briefly explains the sort option(s) used, shows the terminal screenshot of the real output, and gives the resulting answer.

```
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
```

1. Sort according to failed attempts

Command	What it does
<code>sort -k3,3n alerts.txt</code>	The <code>-k3,3</code> flag restricts sorting to only the third whitespace-separated field (failed attempts) and <code>-n</code> tells sort to compare it numerically rather than as text.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ cat alerts.txt
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ sort -k3,3n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ cat suspicious_ips.txt
10.0.0.20
192.168.1.10
10.0.0.8
172.16.0.5

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ sort suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10

```

Fig. 8 — alerts.txt sorted by the third field (failed attempts) plus IP sorting examples.

Question	Answer
Sort according to failed attempts	The records are reordered from the fewest to the most failed attempts: 172.16.0.5 (2), 192.168.1.10 (5), 10.0.0.20 (8), 10.0.0.8 (12).

2. Sort a list of suspicious IP addresses alphabetically

Command	What it does
<code>sort suspicious_ips.txt</code>	Plain sort compares lines byte-by-byte as text (lexicographically), so it does NOT understand that 10.0.0.8 is numerically smaller than 172.16.0.5; it simply compares character by character.

► Output captured together with Fig. 8 above (the same screenshot shows this command run right after the alerts.txt sort).

Question	Answer
Sort a list of suspicious IP addresses alphabetically	Output: 10.0.0.20, 10.0.0.8, 172.16.0.5, 192.168.1.10 — note the lexicographic ordering places 10.0.0.20 before 10.0.0.8 because '2' < '8' at that character position.

3. Display only unique suspicious IP addresses using sort

Command	What it does
<code>sort -u duplicate_ips.txt</code>	The -u (unique) flag removes duplicate lines after sorting, keeping only the first occurrence of each distinct value.

```
gopika@kali: ~/linux/lab

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ cat duplicate_ips.txt
10.0.0.8
192.168.1.10
10.0.0.8
172.16.0.5
192.168.1.10

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ sort -u duplicate_ips.txt
10.0.0.8
172.16.0.5
192.168.1.10

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ cat ports.txt
8080
22
443
21
3306
80

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ sort -n ports.txt
21
22
80
443
3306
8080
```

Fig. 9 — removing duplicate IPs with sort -u, and numeric port sorting.

Question	Answer
Display only unique suspicious IP addresses using sort	Duplicate entries of 10.0.0.8 and 192.168.1.10 are collapsed, leaving a clean unique, sorted list: 10.0.0.8, 172.16.0.5, 192.168.1.10.

4. Sort a list of detected port numbers numerically

Command	What it does
<code>sort -n ports.txt</code>	The -n flag makes sort compare values as numbers instead of as text, which is essential because plain text sort would place “3306” before “443” (comparing the first character '3' vs '4').

► Output captured together with Fig. 9 above (same screenshot, run right after sort -u).

Question	Answer
Sort a list of detected port numbers numerically	Output in correct ascending numeric order: 21, 22, 80, 443, 3306, 8080.

5. Display port numbers in descending numerical order

Command	What it does
<code>sort -rn ports.txt</code>	Combines -n (numeric comparison) with -r (reverse the final order), producing a largest-to-smallest listing.

```
gopika@kali: ~/linux/lab

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ sort -rn ports.txt
8080
3306
443
80
22
21

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ cat usernames.txt
Admin
root
GUEST
admin
Root

(gopika@kali) - [~/linux/lab/sort_lab]
└─$ sort -f usernames.txt
Admin
admin
GUEST
root
Root
```

Fig. 10 — descending numeric port sort and case-insensitive username sort.

Question	Answer
Display port numbers in descending numerical order	Output: 8080, 3306, 443, 80, 22, 21 — the busiest/highest-numbered ports appear first.

6. Sort usernames without considering uppercase/lowercase differences

Command	What it does
<code>sort -f usernames.txt</code>	The -f (fold-case) flag treats uppercase and lowercase letters as equivalent while comparing, so “Admin” and “admin” are grouped together instead of being separated by case.

► Output captured together with Fig. 10 above (same screenshot, run right after the descending port sort).

Question	Answer
Sort usernames without considering uppercase/lowercase differences	Output: Admin, admin, GUEST, root, Root — all variants of the same username (regardless of case) end up next to each other.

7. Count how many times each IP occurs in a failed-login log and show the most frequent first

Command	What it does
<code>sort failed_login.log uniq -c sort -rn</code>	sort first groups identical lines together (a requirement for uniq), uniq -c prefixes each line with its occurrence count, and the final sort -rn orders those counts from highest to lowest.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ cat failed_login.log
10.0.0.8
192.168.1.10
10.0.0.8
10.0.0.8
172.16.0.5
192.168.1.10

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ sort failed_login.log | uniq -c | sort -rn
      3 10.0.0.8
      2 192.168.1.10
      1 172.16.0.5

```

Fig. 11 — counting failed-login attempts per IP and ranking by frequency.

Question	Answer
Count how many times each IP occurs in a failed-login log and show the most frequent first	10.0.0.8 occurred 3 times, 192.168.1.10 occurred 2 times, and 172.16.0.5 occurred once — shown with the busiest attacker IP listed first.

8. Given IP,username,failed_attempts, sort records based on the number of failed attempts

Command	What it does
<code>sort -t, -k3,3n records.csv</code>	-t, sets the comma as the field delimiter (since the file is CSV), and -k3,3n sorts numerically on the third comma-separated field.

```

gopika@kali: ~/linux/lab

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ cat records.csv
192.168.1.10,admin,5
10.0.0.8,root,12
172.16.0.5,guest,2
10.0.0.20,admin,8

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ sort -t, -k3,3n records.csv
172.16.0.5,guest,2
192.168.1.10,admin,5
10.0.0.20,admin,8
10.0.0.8,root,12

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ cat vulnerabilities.csv
CVE-001,SQL Injection,8
CVE-002,XSS,5
CVE-003,RCE,9
CVE-004,Info Disclosure,3

(gopika@kali)-[~/linux/lab/sort_lab]
└─$ sort -t, -k3,3n vulnerabilities.csv
CVE-004,Info Disclosure,3
CVE-002,XSS,5
CVE-001,SQL Injection,8
CVE-003,RCE,9

```

Fig. 12 — sorting a CSV of login records by failed_attempts, and a CSV of vulnerabilities by severity.

Question	Answer
Given IP,username,failed_attempts, sort records based on the number of failed attempts	Rows are reordered from the lowest to the highest failed_attempts value, exactly like the plain-text version but using comma as the delimiter.

9. Sort a CSV vulnerability report based on severity score

Command	What it does
<code>sort -t, -k3,3n vulnerabilities.csv</code>	Identical technique as above, applied to a CVE-ID, description, severity CSV file, sorting on the numeric severity column.

► Output captured together with Fig. 12 above (same screenshot, run right after the login-records CSV sort).

Question	Answer
Sort a CSV vulnerability report based on severity score	Vulnerabilities are listed from least to most severe: CVE-004 (3), CVE-002 (5), CVE-001 (8), CVE-003 (9), making the most critical issue easy to spot at the bottom (or top, with -r added).

10. Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses

Command	What it does
<code>grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log sort -u</code>	grep -oE prints only the parts of each line matching the IPv4 regular expression (instead of the whole line), and piping that into sort -u produces a de-duplicated, sorted list of every distinct IP found in the log.

```

gopika@kali: ~/linux/lab
└─(gopika@kali)-[~/linux/lab/sort_lab]
└─$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10

└─(gopika@kali)-[~/linux/lab/sort_lab]
└─$ awk '{print $1}' failed_login.log | sort | uniq -c | sort -rn | head -5
 3 10.0.0.8
 2 192.168.1.10
 1 172.16.0.5

```

Fig. 13 — extracting IPs from a log with grep and de-duplicating with sort, plus a top-5 frequency pipeline.

Question	Answer
Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses	The pipeline returns each distinct IP address that appears anywhere in access.log exactly once, in sorted order: 10.0.0.20, 10.0.0.8, 172.16.0.5, 192.168.1.10.

11. Create a pipeline that displays the top 5 IPs responsible for the most failed login attempts

Command	What it does
<code>awk '{print \$1}' failed_login.log sort uniq -c sort -rn head -5</code>	awk isolates the IP column, sort groups identical IPs for uniq -c to count, sort -rn ranks the counts highest-first, and head -5 keeps only the top five entries.

► Output: Fig. 13

Question	Answer
Create a pipeline that displays the top 5 IPs responsible for the most failed login attempts	With only three distinct IPs in the sample log, all three are shown ranked by frequency: 10.0.0.8 (3), 192.168.1.10 (2), 172.16.0.5 (1) — on a larger log this same pipeline would return exactly the top five offenders.

Command Cheat Sheet

Command	Use
<code>touch security.txt</code>	Create an empty file
<code>ln security.txt security_hard.txt</code>	Create a hard link
<code>ln -s security.txt security_soft.txt</code>	Create a symbolic (soft) link
<code>ls -li</code>	Show file details and inode numbers
<code>stat security.txt</code>	Display detailed file information
<code>stat security_hard.txt</code>	Display hard-link file information
<code>stat security_soft.txt</code>	Display symbolic-link file information
<code>cat security.txt</code>	Display file contents
<code>echo "text" > security.txt</code>	Write text to a file
<code>echo "text" >> security.txt</code>	Append text to a file
<code>rm security_hard.txt</code>	Remove a hard link
<code>rm security_soft.txt</code>	Remove a symbolic link
<code>sort file.txt</code>	Sort lines alphabetically
<code>sort -r file.txt</code>	Sort lines in reverse order
<code>sort -n file.txt</code>	Sort numbers numerically
<code>sort -u file.txt</code>	Sort and remove duplicate lines
<code>sort -f file.txt</code>	Sort without case sensitivity
<code>sort -k 2 file.txt</code>	Sort using the second field
<code>sort -t ',' -k 2 file.txt</code>	Sort using a comma-separated second field